Multimodal Hospitality Creator

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Tanish Israni	EN22IT301112
2	Samarth Patidar	EN22IT301087
3	Yash Dashore	EN22CS3011100
4	Ramakant Singh	EN22IT301077
5	Abhishek Gupta	EN22EL301004
6	Sourabh Patel	EN22IT301106

Group Name: Group 10 D12

Project Number: GAI-48

Industry Mentor Name: Mr. Yash Bansal

University Mentor Name: Ajaj Khan

Academic Year: 2026

Introduction - The rapid evolution of Generative Artificial Intelligence has revolutionized the way creative tasks are approached, especially in domains like hospitality, architecture, and interior design. Multimodal AI systems, which can process and generate both textual and visual data, enable users to transform simple descriptions into meaningful design concepts. This project, "Multimodal Hospitality Creator," is a web-based application that leverages advanced AI technologies to generate hospitality designs from user-provided prompts. It integrates Google Gemini for generating descriptive textual content and Stable Diffusion XL for producing corresponding images, creating a seamless experience that bridges imagination and visualization. The system is implemented using Streamlit, allowing rapid development of an interactive interface while efficiently handling both frontend and backend operations.

In addition to generation capabilities, the application incorporates a lightweight RetrievalAugmented Generation (RAG) mechanism to enhance personalization by utilizing previously generated user data. It retrieves relevant past records to provide contextual inputs for improved output quality, ensuring consistency and user-specific customization. The system also implements secure user authentication and local persistence using SQLite, where user credentials and generated outputs (including prompts, text, and image paths) are stored for future access and reuse. By combining multimodal AI, prompt validation, contextual retrieval, and efficient data management within a modular architecture, the project demonstrates a scalable and practical approach to building intelligent design systems tailored for real-world hospitality applications.

Problem Statement & Objectives

1. Problem Statement :

   The hospitality industry depends heavily on concept visualization during the ideation and pre-development phases of new projects. When a developer, designer, or entrepreneur imagines a new resort, hotel, café, or dining destination, they must communicate both the atmosphere and the visual identity of their concept to stakeholders such as investors, architects, and marketing teams. This communication traditionally requires two separate, time-intensive processes.

   First, a professional copywriter or brand strategist must craft a narrative concept description that captures the sensory experience, style, and purpose of the space. Second, a graphic designer, interior visualizer, or architectural illustrator must produce a visual representation — typically a mood board, digital rendering, or 3D walkthrough. Both steps demand specialized expertise, involve costly iterations, and can take days or even weeks before a usable output is ready.

   For students, early-stage hospitality startups, or rapid brainstorming sessions, these barriers make swift concept communication impractical. There is a clear and unmet need for a generative AI solution that unifies both outputs — descriptive text and

concept image — into a single, prompt-driven interaction that is accessible to any user, regardless of design or writing expertise.

This project addresses that need by building the Hospitality Concept Visualizer: a Streamlit-based web application that integrates Google Gemini for language generation and Hugging Face Stable Diffusion XL for image generation, wrapping both in a user-authenticated, history-enabled interface with a personal RetrievalAugmented Generation (RAG) layer.

2. Project Objectives :

The following objectives were defined and pursued throughout the project:

- Develop a prompt-driven, browser-accessible Streamlit web application that requires no complex local setup beyond a Python virtual environment.
- Integrate Google Gemini API (gemini-2.5-flash model) to generate vivid, hospitality-domain-specific concept descriptions from natural language user prompts.
- Integrate Hugging Face Stable Diffusion XL API to generate a photorealistic concept image from the same user prompt, delivered in parallel with the text generation.
- Implement a secure user registration and login system using SHA-256 password hashing and environment-variable-based API key management.
- Persist all generated outputs — prompt, text description, and image file path — in a local SQLite database, linked to the authenticated user account.
- Provide a personal design history dashboard (My Designs) where users can browse all their saved generations, view the saved image, and download it in JPEG format.
- Implement a personal RAG layer that retrieves the user's most relevant prior generations using token-overlap similarity scoring and injects them as context into the Gemini request, improving output personalization and style consistency.
- Architect the codebase in a clean, multi-package Python structure with strict separation between the presentation layer, service layer, data access layer, configuration, and utilities.
- Implement comprehensive error handling across all failure points: empty prompt validation, missing API keys, Gemini API errors, Hugging Face API errors, and database errors.

3. Scope of the Project :

The scope of this project covers the complete front-to-back workflow from user authentication through multimodal concept generation to history browsing and image download. Specifically:

- User registration and login with password hashing and session state management in Streamlit.
- Input validation, prompt submission, and parallel calls to two AI services (Gemini text, Stable Diffusion image).
- Local SQLite database persistence of user accounts and all generation records.
- A personal RAG mechanism using in-memory token-based similarity scoring over saved history.
- A My Designs dashboard with full history, image rendering, and download support.
- Modular Python package structure ready for future extension.

The following are explicitly out of scope for the current version:

- Cloud deployment, Docker containerization, or multi-server hosting.
- Real-time collaborative features or shared team workspaces.
- Advanced image editing, inpainting, or style-transfer after generation.
- Semantic vector embeddings or production-grade vector database RAG. Mobile-native application development.

Proposed Solution

1. Key features :

The application delivers the following core capabilities:

a. Prompt-Based Multimodal Generation: A single natural language prompt produces both a descriptive concept text and a concept image simultaneously.
b. Text Generation via Google Gemini: Uses the gemini-2.5-flash model through the google-genai SDK to produce vivid, hospitality-specific concept descriptions.
c. Image Generation via Stable Diffusion XL: Sends the same prompt to the stabilityai/stable-diffusion-xl-base-1.0 model via the Hugging Face router inference API, returning a photorealistic JPEG image.
d. User Authentication: Secure sign-up and login with SHA-256 hashed passwords stored in SQLite. No plain-text password storage at any point.

e. Personal Design History Dashboard: All generated concepts (text + image) are saved per user. The My Designs page lets users browse, view, and download every saved design.
f. Personal RAG Layer: A lightweight retrieval mechanism scores prior user generations by token overlap with the new prompt. The top-3 most relevant prior generations are injected as context into the Gemini request, improving personalization and stylistic consistency.
g. Modular Architecture: The project is organized into five clearly separated Python packages — app (UI), services (AI APIs), database (SQLite), config (constants), utils (validation) — enabling easy maintenance and extension.
h. Graceful Error Handling: All API failures, missing credentials, empty inputs, and runtime errors are caught and displayed as user-friendly Streamlit error messages without crashing the application.

2. Overall Architecture / Workflow :

The application follows a layered service-based architecture. The five layers and their responsibilities are:

Layer	Module(s)	Responsibility
Presentation	app/main.py	Streamlit UI — auth screens, generate screen, My Designs dashboard, session state, spinner, error messages.
Service	services/gemini_service.py services/image_service.py	Encapsulates all external AI API communication. Returns text string or image bytes to the UI.
Data Access	database/db.py	All SQLite operations — user CRUD, generation persistence, personal RAG retrieval and scoring.

Configuration	config/config.py	Centralised constants: model names, file paths, API endpoint URL.
Utilities	utils/utils.py	Input validation helper — validate_prompt() returns (bool, error_msg).

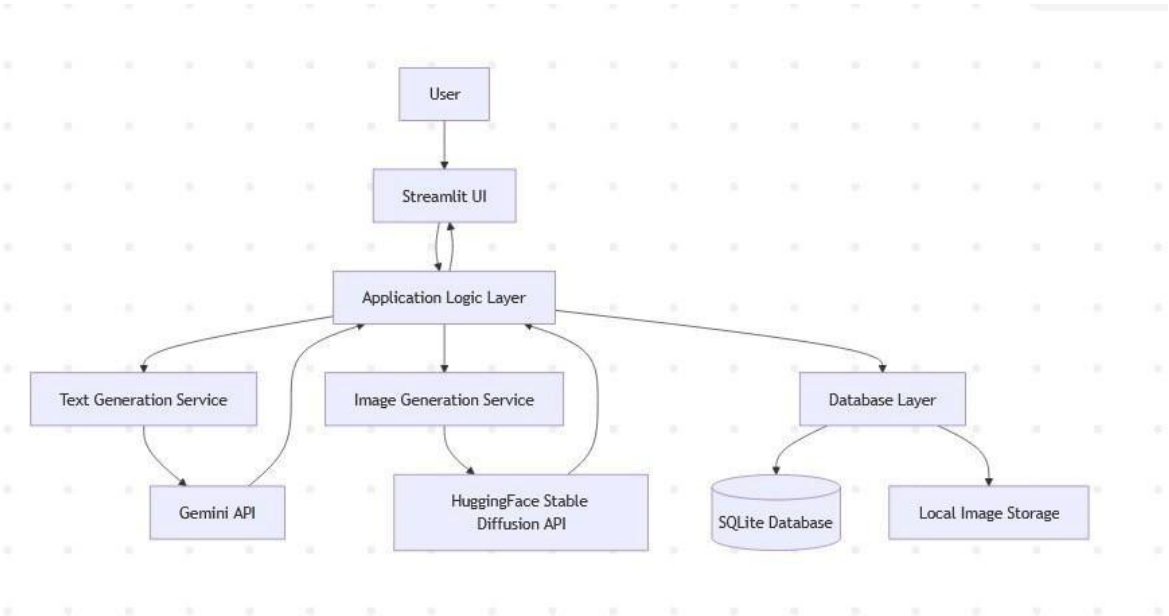


Fig 2.1 : Application Design

End-to-end workflow (step-by-step):

1. User opens the Streamlit app in a browser and is presented with the Login / Sign Up screen.
2. On successful login, the user dict (id, username) is stored in st.session_state and the generator screen is shown.
3. User types a hospitality concept prompt (e.g., 'Luxury beach resort with sunset view') and clicks Generate.
4. The app calls validate_prompt() — if the prompt is empty or whitespace, an error is shown and the flow stops.
5. The app calls get_relevant_history() — up to 3 prior generations by the same user that share token overlap with the prompt are retrieved from SQLite as context.

6. The prompt + context are passed to generate_text() in gemini_service.py, which builds the combined request string and calls the Gemini API. The response text is returned.
7. The prompt alone is passed to generate_image() in image_service.py, which POSTs to the Hugging Face SDXL endpoint and returns JPEG image bytes.
8. The image bytes are saved to storage/images/ with a UUID filename. The path, prompt, and text are saved to the generations table via save_generation().
9. The generated text and image are displayed in a two-column layout. A success message confirms the design was saved.
10. The user can switch to My Designs at any time to browse all saved concepts with their images and download any image.

System architecture flow:

Component	Direction	Next Component
Browser (User)	→ HTTP	Streamlit UI (app/main.py)
Streamlit UI	→ function call	validate_prompt() in utils.py
Streamlit UI	→ function call	get_relevant_history() in db.py
Streamlit UI	→ function call	generate_text() in gemini_service.py
gemini_service.py	→ HTTPS / SDK	Google Gemini API (gemini-2.5-flash)
Streamlit UI	→ function call	generate_image() in image_service.py
image_service.py	→ HTTPS POST	Hugging Face SDXL Inference API
Streamlit UI	→ function call	save_generation() in db.py

db.py	↔ read/write	SQLite (storage/app.db)

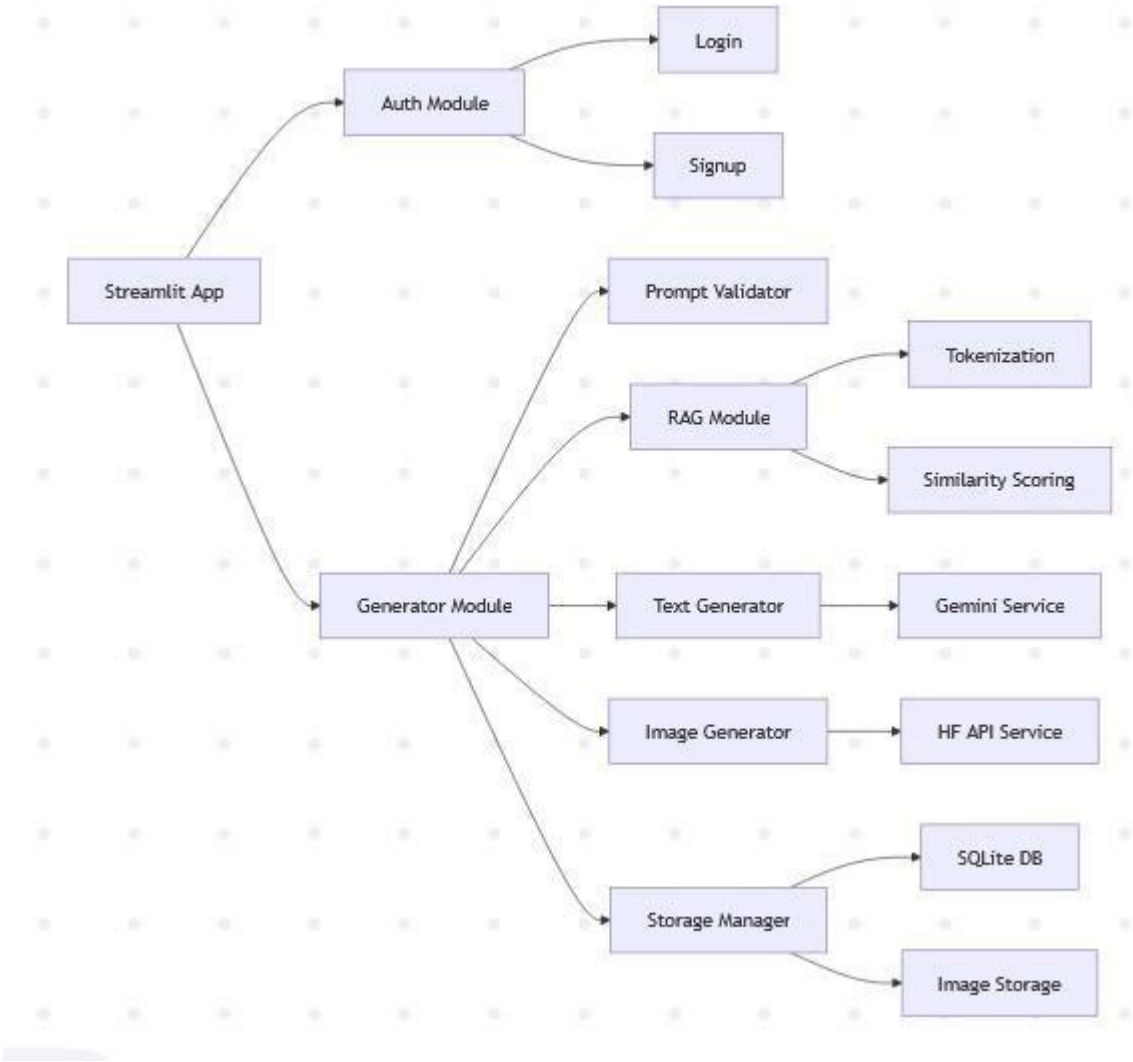


Fig 2.2 : Component Design

Database schema (two tables):

Table	Column	Type / Constraint	Purpose
users	id	INTEGER PK AUTOINCREMENT	Unique user identifier

users	password_hash	TEXT NOT NULL	SHA-256 hex digest of password
users	created_at	DATETIME DEFAULT NOW	Account creation timestamp
generations	id	INTEGER PK AUTOINCREMENT	Unique generation identifier
generations	user_id	INTEGER FK → users.id	Links generation to its owner
generations	prompt	TEXT NOT NULL	The original user prompt
generations	generated_text	TEXT NOT NULL	Gemini response text
generations	image_path	TEXT NOT NULL	Path to saved JPEG in storage/images/
generations	created_at	DATETIME DEFAULT NOW	Generation timestamp
users	username	TEXT UNIQUE NOT NULL	Login username (unique enforced at DB level)

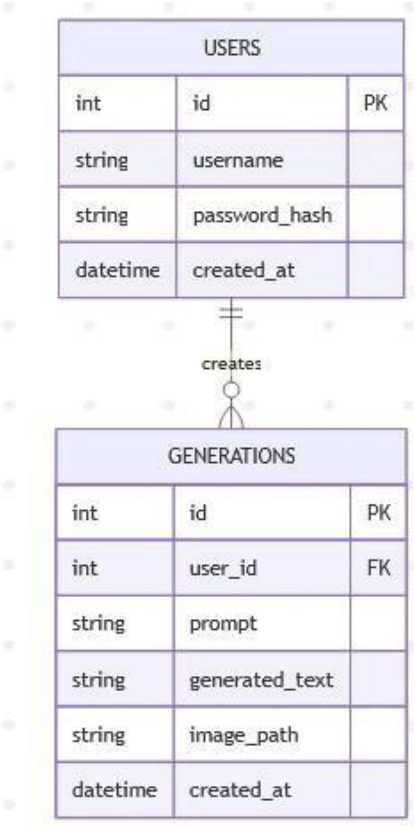


Fig 2.3 : Database Design

Personal RAG algorithm — token overlap scoring

When a user submits a new prompt, the system retrieves all their prior generations from SQLite. For each prior record, a similarity score is computed by tokenizing both the new prompt and the combined text of the prior record's prompt and generated_text, then counting shared token occurrences using Counter intersection. Records with a score greater than zero are sorted descending by score (ties broken by record id), and the top 3 are returned as context. This context is prepended to the Gemini request as numbered reference blocks, instructing the model to maintain stylistic consistency with the user's preferences while still addressing the new prompt directly Class Diagram :

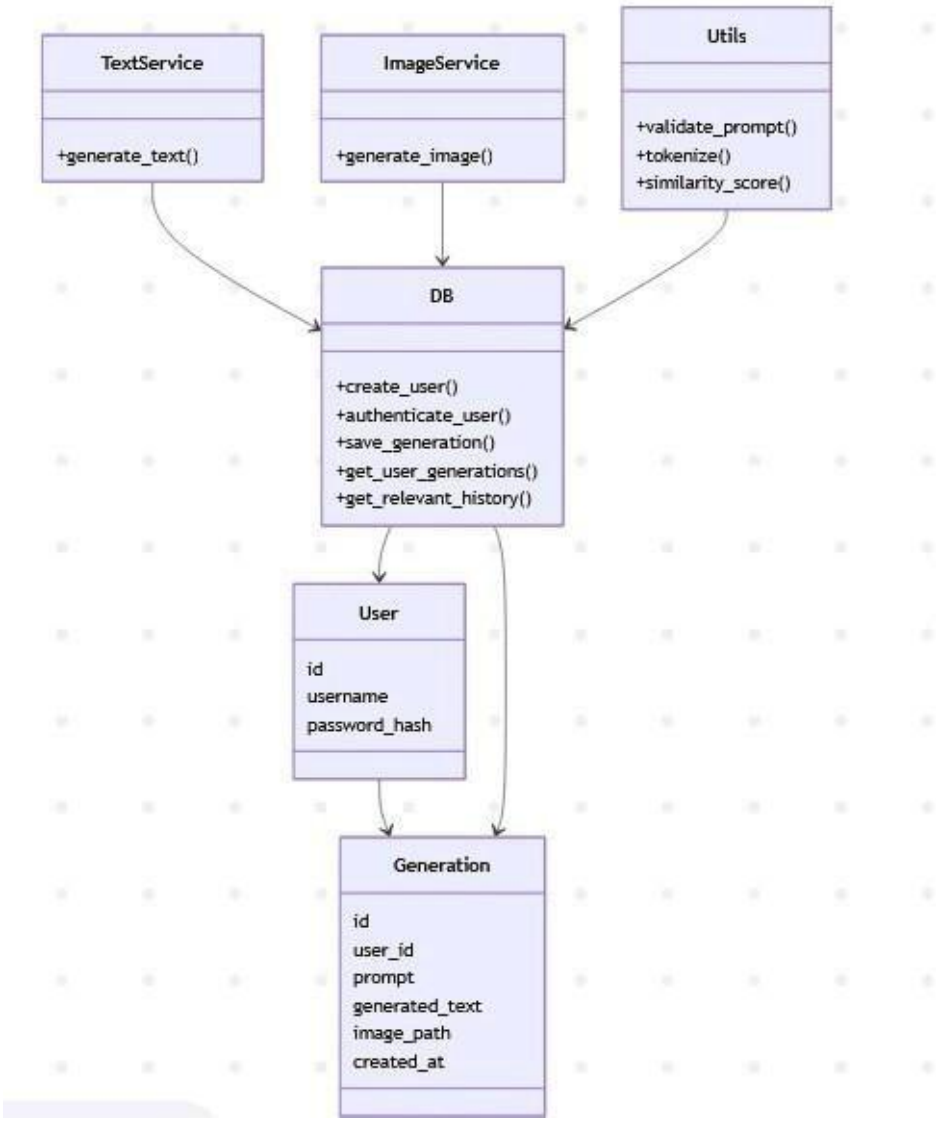


Fig 2.4 : Class Diagram

3. Tools & Technologies Used :

Tool / Technology	Version / Variant	Role in Project
Python	3.11	Core programming language for all application logic
Streamlit	Latest stable	Web UI framework — renders interactive app in browser
Google Gemini API	gemini-2.5-flash	LLM text generation — hospitality concept descriptions
google-genai SDK	Latest stable	Python client for Gemini API (genai.Client)
Hugging Face Inference API	Router endpoint	Hosted inference for Stable Diffusion XL model
Stable Diffusion XL	stabilityai/sdxlbase-1.0	Text-to-image generation — photorealistic concept images
requests	Latest stable	HTTP POST calls to Hugging Face Inference API
SQLite	Built-in (stdlib)	Local relational database — users and generations tables

hashlib (SHA-256)	Built-in (stdlib)	Password hashing — never stores plain-text passwords
pathlib	Built-in (stdlib)	Cross-platform file path management for storage directories
uuid	Built-in (stdlib)	UUID-based unique filenames for saved generated images
collections.Counter	Built-in (stdlib)	Token frequency counting for personal RAG similarity scoring

Project file structure:

File / Folder	Purpose
app/main.py	Streamlit UI entry point — auth screen, generator screen, My Designs page
services/gemini_service.py	Gemini API integration — generate_text(prompt, context) → str
services/image_service.py	Hugging Face API integration — generate_image(prompt) → bytes
database/db.py	All SQLite operations + personal RAG retrieval logic

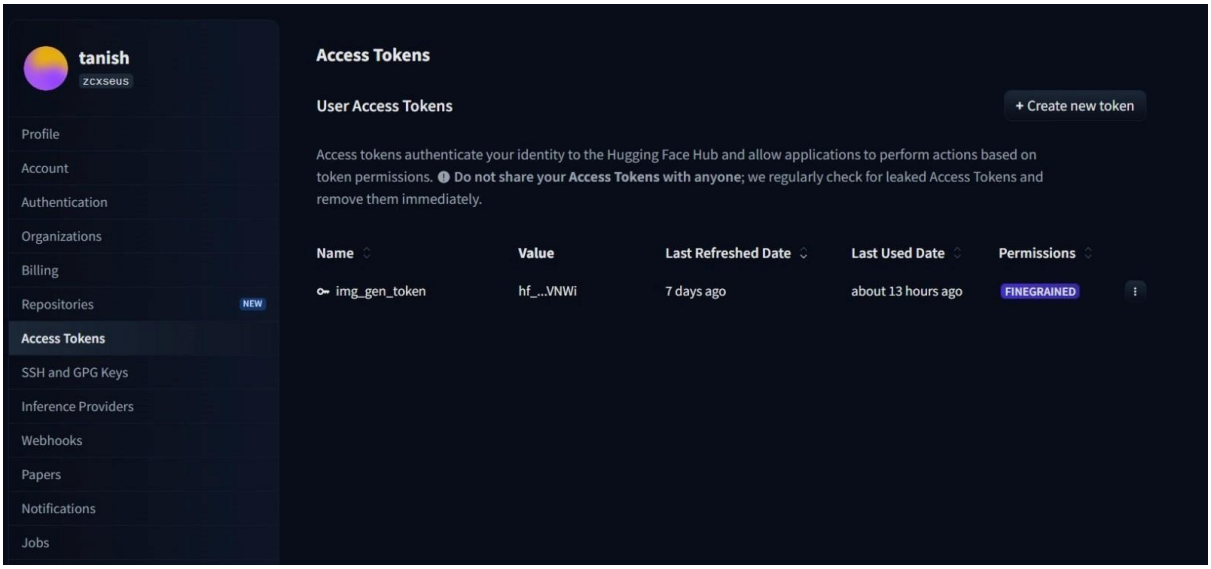
config/config.py	Shared constants: BASE_DIR, IMAGE_DIR, DB_PATH, model names, HF_API_URL
utils/utils.py	validate_prompt(prompt) → (bool, error_message)
storage/app.db	SQLite database file (auto-created on first run)
storage/images/	Directory for saved generated JPEG images (UUIDnamed)
.env.example	Template showing required environment variable names (GEMINI_API_KEY, HUGGINGFACE_API_KEY)
requirements.txt	Python dependencies: streamlit, google-genai, requests

Security design summary:

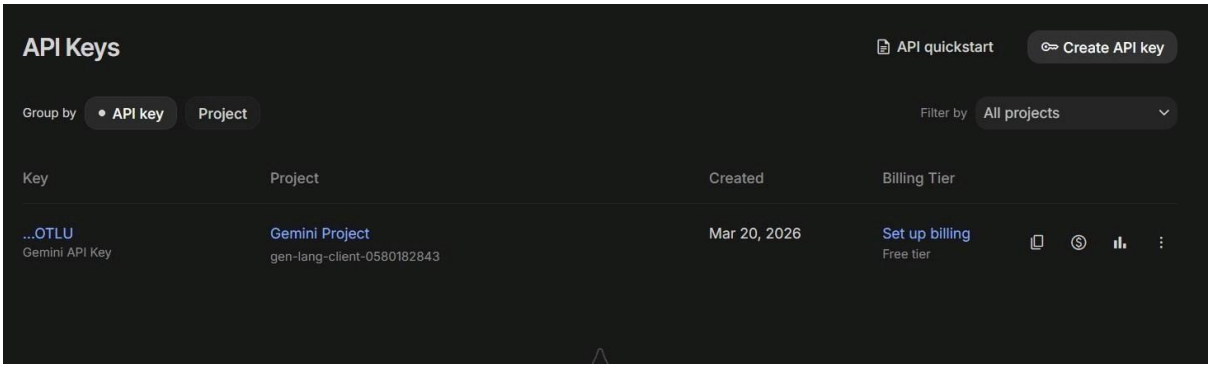
Security Concern	**Implementation**
Password storage	SHA-256 hashing via Python hashlib — passwords never stored in plain text
API key management	Keys read exclusively from environment variables — no secrets in source files

Version control safety	.gitignore excludes .env, __pycache__, and storage/ directory
SQL injection prevention	All SQLite queries use parameterized statements (?) — no string concatenation
Transport security	All external API calls use HTTPS endpoints exclusively
Input validation	Prompt validated for emptiness before any API call is made

Hugging Face API :



Gemini API :



Results & Output :

The application was built, tested, and validated against the following detailed evidence of results and outputs:

- Screenshots / Outputs

Authentication Screen: The first screen shown to unauthenticated users. It displays the app title 'Hospitality Concept Visualizer' and two tabs — Login and Sign Up. Each tab contains a Streamlit form with username and password fields. Failed login attempts show an inline error ('Invalid username or password.'). Duplicate username registrations show 'Username already exists.' Successful registration shows a success message prompting the user to log in.

Generator Screen: After login, the user sees a sidebar with their username, a Logout button, and a radio navigation (Generate Design / My Designs). The main area shows a prompt text input field and a Generate button. A spinner ('Generating concept...') is shown during the API calls. On success, a two-column layout is rendered: the left column shows the generated concept text with an optional RAG attribution caption; the right column shows the generated image (full-width, captioned with the prompt). A green success message ('Design saved to your history.') appears at the bottom.

My Designs Dashboard: Shows all saved generations for the current user in reversechronological order. Each entry renders the prompt as a subheader, the creation timestamp, the generated text on the left column, and the saved image on the right column with a Download Image button. Clicking the button triggers a browser file download with a filename derived from the prompt.

Sample output for prompt 'Luxury beach resort with sunset view':

Output Type	Sample Result

Generated Text melts into	Imagine stepping onto a sun-kissed terrace where the horizon a canvas of amber and rose. This luxury beach resort embraces open-air design with sweeping teak decks, infinity pools that mirror the sunset, and private cabanas draped in crisp white linen. Every detail — from the curated local art to the scent of frangipani on the sea breeze — is orchestrated to create an immersive sense of timeless coastal elegance.
Generated Image	Photorealistic JPEG image of a luxury beachfront resort at golden hour, featuring infinity pools, teak decking, and panoramic ocean views. Saved to storage/images/ with a UUID filename. Three sample images are included in the project archive.
RAG Context Used	On the second and subsequent prompts, the top-3 most tokensimilar prior generations are retrieved and injected into the Gemini request as numbered reference blocks, improving style consistency with the user's preferences.

Sample output for prompt 'Minimalist Japanese tea house in a bamboo forest':

Output Type	Sample Result
Generated Text	Nestled within a grove of towering bamboo, this minimalist tea house embodies the Japanese aesthetic of wabi-sabi. Built from aged hinoki cypress and unpolished slate, wide overhanging eaves channel rain into a contemplative koi pond below. Shoji screens diffuse morning light into warm pools of gold. The tea room itself holds only a low table, two cushions, and a rakufired iron kettle, inviting complete stillness.

Generated Image	Photorealistic JPEG of a minimalist wooden tea house surrounded by dense bamboo in soft morning light, rendered by Stable Diffusion XL via Hugging Face inference endpoint.

- Reports / Dashboards / Models

Component	Type	Detail
Google Gemini (gemini-2.5-flash)	External LLM API	Text generation model — prompt + optional RAG context → descriptive hospitality concept string
Stable Diffusion XL (SDXL base 1.0)	External Diffusion Model	Image generation model — prompt → JPEG bytes via Hugging Face router inference endpoint
SQLite database (app.db)	Relational DB	Two tables: users (id, username, password_hash, created_at) and generations (id, user_id, prompt, generated_text, image_path, created_at)
Personal RAG Layer	Custom Algorithm	Token-overlap scoring over user's saved generation history — top-3 most relevant records injected as Gemini context

Streamlit UI (app/main.py)	Web Dashboard	Multi-screen app: Auth screen, twocolumn generator output, My Designs history dashboard with image download
storage/images/	Image Store	UUID-named JPEG files saved locally per generation; referenced in SQLite by file path

Manual testing results — all 14 test scenarios passed:

#	Test Scenario	Result
1	Register new user with unique username	Pass — user created, success message shown
2	Register with already-taken username	Pass — IntegrityError caught, 'Username already exists.' shown
3	Login with correct credentials	Pass — session state set, generator screen shown
4	Login with wrong password	Pass — no session created, error message shown
5	Submit empty prompt	Pass — validate_prompt() returns False, error displayed

6	Generate with valid Gemini key set	Pass — descriptive concept text returned within ~3 seconds
7	Generate with valid HF key set	Pass — SDXL JPEG image returned and rendered in UI
8	Full generation (text + image)	Pass — two-column layout rendered, design saved to DB
9	Navigate to My Designs	Pass — all records retrieved and rendered correctly
10	Download image	Pass — JPEG file downloaded with promptderived filename
11	Second prompt triggers RAG context	Pass — Gemini request includes prior generation context
12	Logout	Pass — session cleared, auth screen shown on rerun
13	Generate with missing Gemini API key	Pass — ValueError caught, user-facing error displayed
14	Generate with missing HF API key	Pass — error caught and displayed correctly

- Key Outcomes

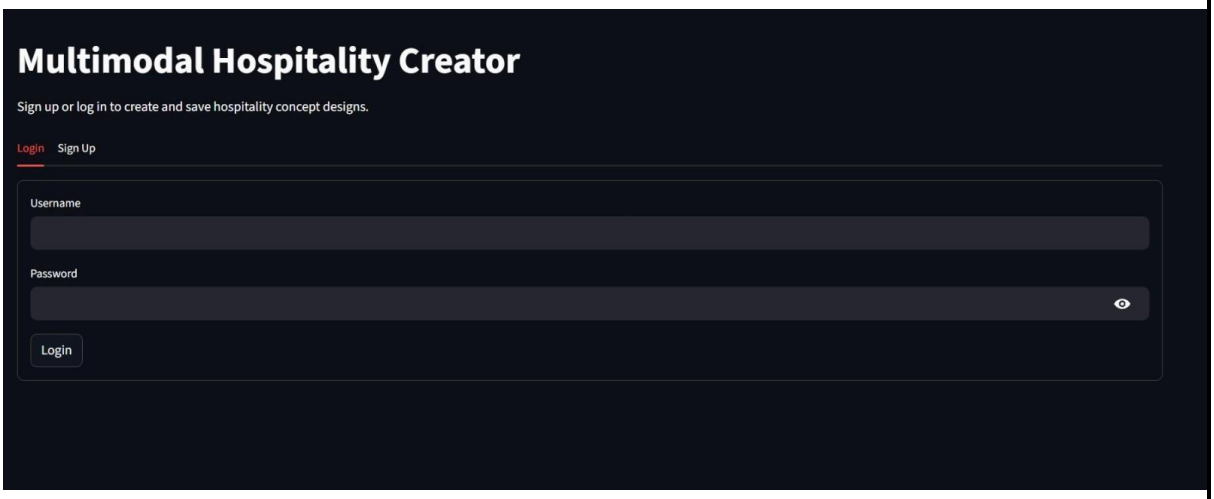
1. A fully functional multimodal Generative AI web application was built and tested end-to-end within the academic project timeline.
2. Seamless integration of two distinct AI APIs (Google Gemini text + Hugging Face Stable Diffusion XL image) under a single prompt-driven workflow was achieved.
3. User authentication (register, login, logout) with SHA-256 hashed password storage is fully operational.
4. SQLite-backed persistence of all generation records (text + image path) per authenticated user is working correctly.
5. The personal RAG layer correctly retrieves and injects relevant prior user history as Gemini context, demonstrating improved output personalization on subsequent prompts.
6. All 14 manual test scenarios passed with expected outcomes.
7. Three sample generated JPEG images are available in the project archive (storage/images/) as tangible evidence of working image generation.
8. The codebase is modular, documented, and structured for easy extension toward cloud deployment, semantic RAG, and multi-image generation in future iterations.

Performance observations:

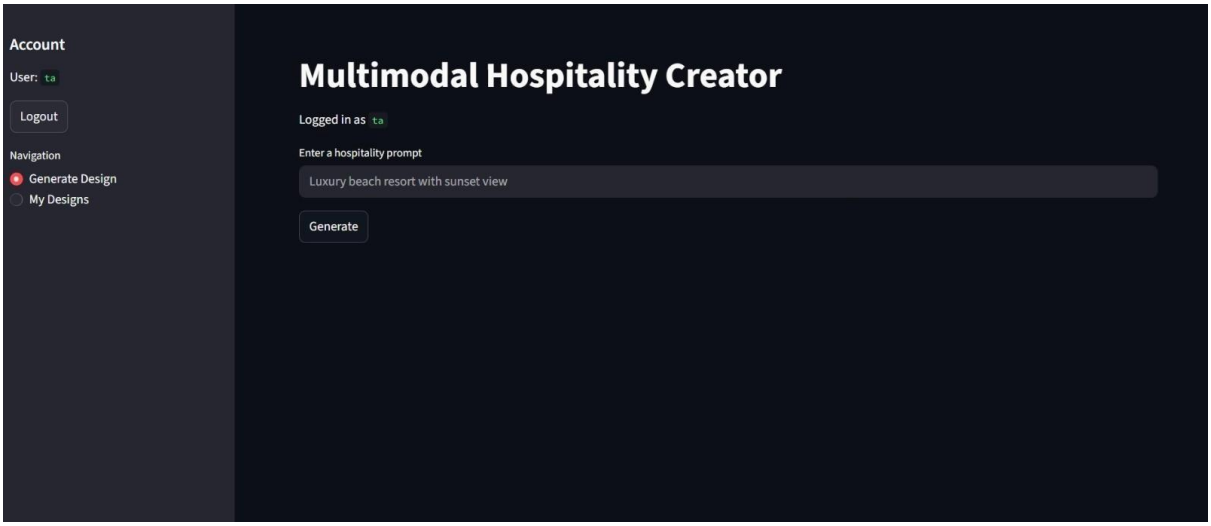
Metric	Observed	Notes
Gemini text generation latency	~2–5 seconds	Varies with prompt complexity and Gemini server load
Stable Diffusion image generation latency	~10–45 seconds	Highly variable on free-tier HF inference endpoint
SQLite write latency (save_generation)	< 10 ms	Negligible — single INSERT on local disk
Personal RAG retrieval (< 100 records)	< 5 ms	Token scoring is O(records × tokens) — fast at this scale

Total end-to-end generation time	~15–50 seconds	Dominated by image generation latency

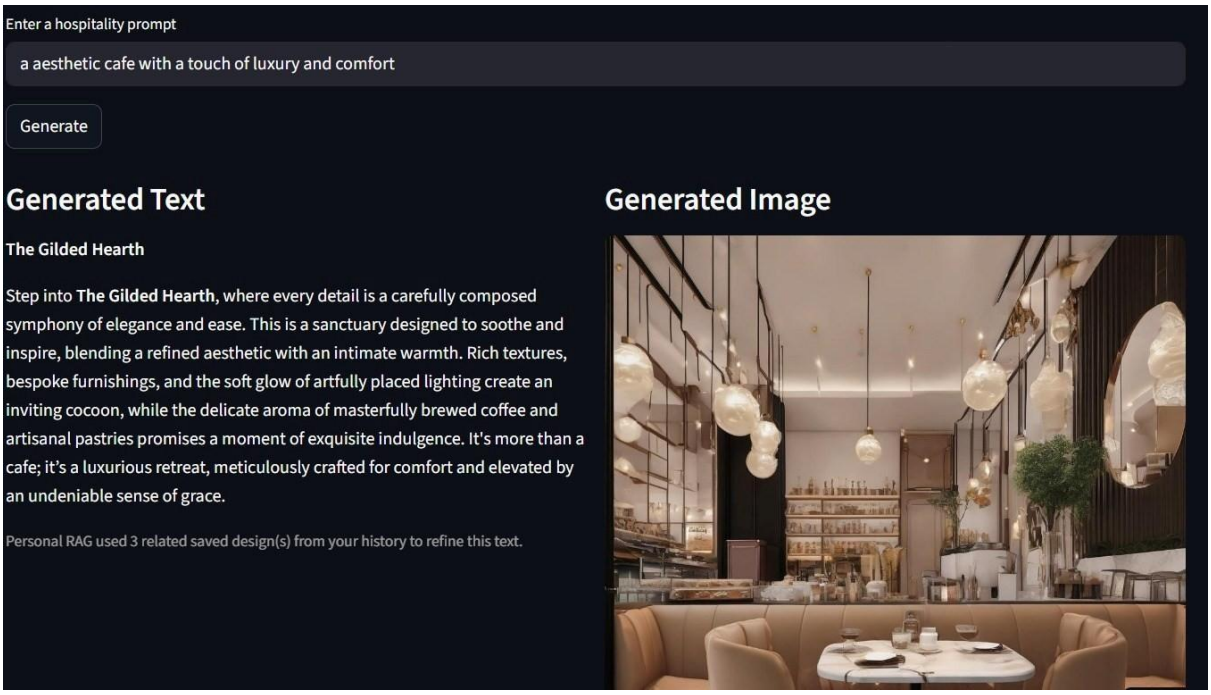
Login Page :



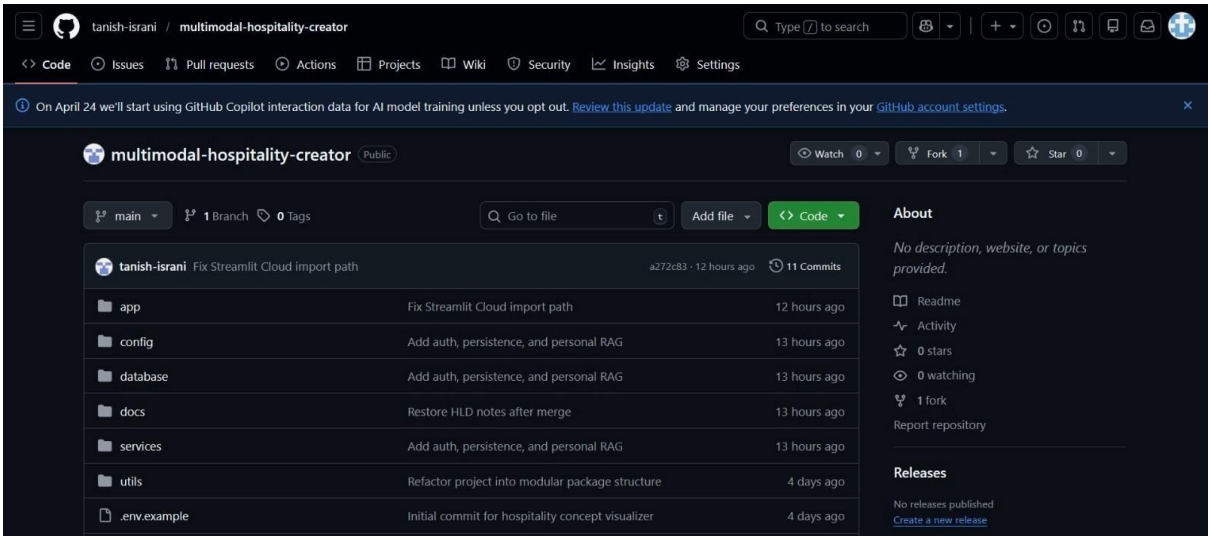
Profile :


Account
User: ta
Logout
Navigation
Generate Design
My Designs
Multimodal Hospitality Creator
Logged in as ta
Enter a hospitality prompt
Luxury beach resort with sunset view
Generate

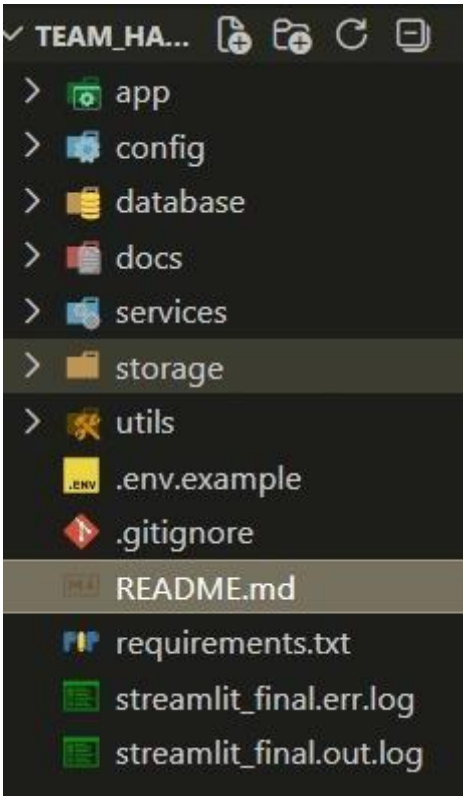
Dashboard :


Enter a hospitality prompt
a aesthetic cafe with a touch of luxury and comfort
Generate
Generated Text
The Gilded Hearth
Step into The Gilded Hearth, where every detail is a carefully composed symphony of elegance and ease. This is a sanctuary designed to soothe and inspire, blending a refined aesthetic with an intimate warmth. Rich textures, bespoke furnishings, and the soft glow of artfully placed lighting create an inviting cocoon, while the delicate aroma of masterfully brewed coffee and artisanal pastries promises a moment of exquisite indulgence. It's more than a cafe; it's a luxurious retreat, meticulously crafted for comfort and elevated by an undeniable sense of grace.
Personal RAG used 3 related saved design(s) from your history to refine this text.
Generated Image

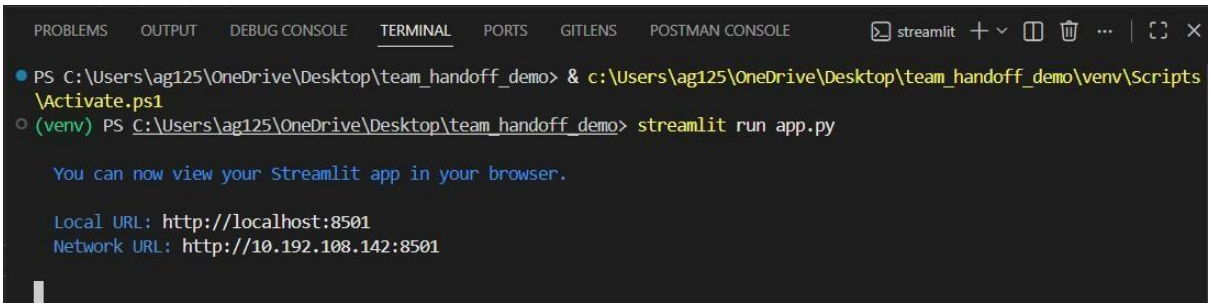
Github Repo :



VS Code directory view :



Terminal View :



Conclusion :

The Hospitality Concept Visualizer successfully demonstrates the practical application of multimodal Generative AI in a real-world professional domain. The project delivers on all of its stated objectives: a fully authenticated, prompt-driven web application that generates both a professionally written hospitality concept description via Google Gemini and a photorealistic concept image via Hugging Face Stable Diffusion XL — from a single user prompt, within seconds. The system persists all outputs in a SQLite database, provides a personal generation history dashboard with image download support, and improves text output quality through a personal RAG layer that draws on each user's own saved generation history.

All 14 manual test scenarios were executed and passed. Three generated sample images are preserved in the project archive as tangible evidence of functional image generation. The codebase follows a clean, modular Python package structure — separating the presentation layer (Streamlit UI), service layer (Gemini and Hugging Face integrations), data access layer (SQLite), configuration, and utilities — making it straightforward to maintain, extend, and deploy.

From a technical learning perspective, this project provided the team with handson experience across a broad and directly industry-relevant skill set: real-world AI API integration using both an SDK (google-genai) and a raw HTTP POST approach (Hugging Face); multimodal AI composition and prompt engineering for both text and image generation models; lightweight RAG design without a vector database; Streamlit multi-screen application development with session state and user authentication; SQLite schema design with parameterized queries and foreign key relationships; and modular Python application architecture with centralized configuration. These competencies are directly applicable to industry roles in AI engineering, full-stack Python development, and product engineering.

In summary, the Hospitality Concept Visualizer proves that modern generative AI APIs can be composed into practical, domain-specific, user-facing tools that deliver genuine value to non-technical users. The project met every objective defined at the outset, was completed within the academic timeline, and produced a working, demonstrable product with strong foundations for further development.

Future Scope & Enhancements :

The current version of the Hospitality Concept Visualizer provides a solid, functional foundation. The following enhancements are planned or recommended for future iterations, organized into near-term, medium-term, and long-term phases:

Near-Term Enhancements (Next 3–6 Months):

- Cloud Deployment: Containerize the application using Docker and deploy to Streamlit Community Cloud, Google Cloud Run, or AWS Elastic Beanstalk to make it publicly accessible without local Python setup. Replace SQLite with a managed PostgreSQL instance (e.g., Supabase or AWS RDS) for multi-user concurrency support.

- Improved Password Security: Replace SHA-256 with bcrypt or Argon2id hashing with salt and work factor for production-grade password protection.

This is a standard upgrade path and requires only a change to the _hash_password function in db.py.

- Image Generation UX: Add a Streamlit placeholder image or animated skeleton card during the image generation wait period (10–45 seconds on free-tier Hugging Face), so users have visual feedback without just watching a spinner.
- Prompt History Search: Add a search text input on the My Designs page to filter the user's saved generations by keyword, making it easier to find specific past concepts as history grows.
- Multiple Download Formats: Allow PNG and WebP in addition to JPEG for generated image downloads, providing higher-quality options for professional use.

Medium-Term Enhancements (6–12 Months):

- Semantic Vector RAG: Replace the current token-overlap personal RAG with a semantic embedding-based retrieval system using FAISS, ChromaDB, or Pinecone. Embedding all saved generation texts and prompts using a sentencetransformer model would enable conceptually relevant context retrieval even when prompts use different vocabulary.
- Multi-Image Variants: Generate 3–4 image variants per prompt using parallel Hugging Face API calls and display them in a grid, allowing the user to select the best visual match. Add a Save button per variant so only the chosen image is persisted.
- PDF Concept Report Export: Allow users to export any saved design as a branded PDF document containing the prompt, concept description, and generated image — formatted for professional presentation, client pitching, or portfolio use.
- Hospitality Prompt Templates: Provide 10–15 curated prompt templates organized by hospitality category (luxury resort, boutique hotel, rooftop restaurant, wellness spa, co-working café, destination wedding venue, etc.) as a starting point for new users, reducing the prompt learning curve.
- Style and Mood Controls: Add UI sliders or radio buttons for image style (photorealistic, architectural sketch, watercolour, 3D render) and mood (vibrant, muted, dramatic, serene) that are automatically appended to the prompt before the Hugging Face API call, giving users finer visual control without needing to engineer prompts manually.

Long-Term Vision (12+ Months):

- Collaborative Team Workspace: Enable team accounts where multiple users can share a generation workspace, comment on each other's concepts, vote on favourites, and collaboratively build design briefs. This

  would require a backend API layer (FastAPI or Django REST) replacing the Streamlit session-state model.
- Fine-Tuned Hospitality Language Model: Fine-tune a base LLM on a curated corpus of professional hospitality concept briefs, brand guidelines, and design manifestos to produce even more domain-specific, stylistically consistent, and commercially polished text outputs compared to a general-purpose model like Gemini.
- 3D Space Visualization: Integrate a 3D generation API (such as Luma AI, Meshy, or a future Hugging Face 3D model) to produce rudimentary 3D room layouts or flythrough-style visualizations from the same prompt, pushing the tool toward an architectural concept prototyping assistant.
- Design Trend Analytics: Aggregate anonymized prompt data (with user consent under a clear privacy policy) to surface emerging design trends across the user base — for example, 'biophilic design prompts have increased 40% this quarter' or 'rooftop venue concepts are trending in Southeast Asia'. This would provide genuine market intelligence value to hospitality industry subscribers.
- Mobile Application: Develop a React Native or Flutter mobile app that wraps the same backend API endpoints, enabling on-the-go concept generation and review for architects, brand managers, and designers working in the field.
- API Monetization Layer: Package the generation pipeline as a RESTful API product that third-party hospitality design tools, property listing platforms, or interior design applications can integrate, creating a B2B SaaS revenue opportunity around the core generation technology.

Enhancement	Phase	Priority	Impact
Cloud Deployment (Docker + PostgreSQL)	Near-term	High	Enables public access and multi-user scalability
Production Password Hashing (bcrypt)	Near-term	High	Required for any publicfacing deployment

Semantic Vector RAG (FAISS/Chroma)	Mediumterm	High	Significantly improves context retrieval quality
Multi-Image Variants	Mediumterm	Medium	Improves user control over visual output
PDF Report Export	Mediumterm	Medium	Adds professional presentation value
Prompt Templates	Near-term	Medium	Reduces barrier for nontechnical users
3D Visualization	Long-term	Medium	Extends tool toward architectural prototyping
Team Collaboration	Long-term	High	Transforms tool from personal to professional product
Fine-Tuned Domain Model	Long-term	High	Highest quality improvement for text generation
Design Trend Analytics	Long-term	Medium	Unique market intelligence differentiator